Ex: 6 ERROR CORRECTION AND DETECTION USING HAMMING CODE

Aim:-

To simulate hamming code data to detect and correct it.

PROCEDURE:- ALGORITH

* Start the program
* Read the number of bits) bits
* Using the hamming code concept find redundant bits assuming even parity
* Add the redundant bits in the data at according places.
* Send the data to receiver side
* Find the redundant bits again
* Take 1's complement
* If all 0 show the decoded data else reverse the output and convert it to int and flip the bit in that position.
* Stop the program.

code:

```
def calculate_parity_bits(data):
    m = len(data)
    r = 0
    while(2**r) < (m + r + 1):
        r += 1
    return r
```

```
def generate_hamming_code(data):
    M = len(data)
    r = calculate_parity_bits(data)
    n = M + r
    hamming = ['0'] + [n+1]
    J = 0
    for i in range(1, n+1):
        if range(1, n+1):
            hamming[i] = data[i]
            J+1
    for r in range(r):
        position = 2**p
        parity = 0
        for r in range(, n+1):
            if j position:
                parity ^= int(hamming[i].
        hamming[position] = str(parity)
    return ''.join(hamming[1:1])

def defect_and_correct(recived):
    n = len(recived)
    r = 0
    while(2**r) < (n+1):
        r += 1
    hamming = ['0'] + list(recived)
    error_position = 0
    if error_position != 0:
        print("Error. position", error_position)
        if hamming[error_position] == '0':
            hamming[error_position] = 1:
```

Output:

Enter the binary data : 1011
original Data : 1011
Hamming code : 0110011
Enter the recived Hamming code : 0110111
Enter position : 5
Error deleted at position 5
Error corrected Successfully !
Recived code : 0110111
Corrected Code : 0110011

```
else:
    hamming[error_position] = '0'.
return ''.join(hamming[1:])
Print(" Hamming code')
data = input("Enter the binary data: ")
if not all(bit in '01' for bit in data):
        print("please enter only 0 and 1")
else:
    hamming_code = generate_hamming_code(data)
    print("Hamming code", hamming code)
if len(recived) != len(hamming_code) or
        not all(bit 'in '01' bit in recived):
    else:
        corrected code = detect_and_correct(recive)
```

Result :

The Hamming Ode program was succesfully implemented in ~~Error~~ Python.